

SentinelAML: Study Pack

Architecture, measured results, reverse-engineering guide, interview preparation and resume evidence

September 2026

Contents

Project overview	2
The business problem	2
Headline results (measured, synthetic data)	2
Architecture at a glance	3
Run it locally	3
Implemented locally vs production architecture	3
Repository map	4
Documentation	4
Architecture	4
1. Problem framing	4
2. Why each kind of intelligence is used, and where it is not	5
3. Component view	5
4. Communication and state	5
5. Scaling: hundreds to millions	6
6. Cost model and optimisation	7
7. Known gaps and deliberate trade-offs	7
Evaluation results	7
1. Data pipeline	7
2. Machine learning (alert risk model)	8
3. Retrieval (RAG)	8
4. Agentic workflow	9
5. System performance	9
6. Findings that changed the system	10
Reverse-engineering guide	10
Level 0: The business, in plain English	10
Level 1: Run it and watch	11
Level 2: Component by component	12
Level 3: Trace one alert through the code	15
Level 4: Break it on purpose	15
Level 5: Senior-engineer view	16
14-day plan (about 4-hour sessions)	16
Interview preparation: SentinelAML	16
Basic	17
Intermediate	22
Advanced	27
Resume evidence	34
1. What was built (verifiable in code)	34

2. Measured results	35
3. Engineering judgement (true stories for interviews)	35
4. Do NOT claim	35
5. Example bullets (edit to your voice; keep the qualifiers)	36

Project overview

SentinelAML helps financial-crime teams at Irish and EU banks and payment firms clear transaction-monitoring alerts faster **without removing human accountability**.

The business problem

Rules-based transaction monitoring is a regulatory control every bank must run. It is tuned to miss nothing, so most alerts are false positives. Investigators spend hours per alert pulling transactions, checking customer profiles, reading policy and writing a narrative. Genuine cases wait in the same queue as cash businesses lodging their weekend takings.

SentinelAML keeps the rules and changes what happens after an alert fires.

Step	Technology	Why this technology
Rank every alert by risk	XGBoost + Platt calibration, Isolation Forest	Structured numeric data; needs a calibrated probability and explanations, not text generation
Classify and route the alert	Claude Haiku-class model	Cheap, fast extraction and tagging
Gather evidence	Claude Sonnet-class agent with read-only, case-scoped tools	Multi-step planning over several data sources
Find applicable policy and regulation	Hybrid RAG (pgvector + BM25 + rerank) with access control	Knowledge lives in documents, changes often, and is confidential by role
Recommend an action and draft the narrative	Claude Sonnet-class model	Synthesis and reasoning across evidence, scores and policy
Check every claim	Deterministic verifier	The LLM is untrusted; verification must be reproducible
Decide if a human must approve	Deterministic policy engine	Legal authority cannot be delegated to a model
Execute approved actions, keep the trail	Allow-listed executor, hash-chained audit log	Only code paths that change outcomes; tamper-evident history

Headline results (measured, synthetic data)

Full tables and caveats are in [docs/evaluation.md](#).

- **ML, held-out future period:**
 - XGBoost PR-AUC **0.976** vs logistic baseline **0.958** (identical protocol).
 - 91.7% recall with 0 false positives at the chosen threshold.
 - **73.7%** of alerts can be routed to lighter review.
- **Retrieval, 36 labelled queries:**
 - Hybrid + rerank: Recall@5 **0.903**, MRR **0.801**, nDCG@5 **0.807**.
 - Dense-only: Recall@5 0.736.
- **Access control:** 21,147 retrieved results audited, **0** unauthorised.
- **Agent workflows (160 held-out alerts):**
 - 100% task success.
 - 735 numeric claims re-computed from the database, **0** mismatches.
 - **0** dangerous actions executed without approval.
 - **0** suspicious alerts auto-closed.
- **Adversarial testing:** 96 fault-injection runs (timeouts, loops, hallucinated evidence, fabricated citations, PII leaks, unsafe closures): **100%** failed safe.
- **Tests:**
 - 134 passing on PostgreSQL + pgvector.
 - 129 passing on SQLite, plus 5 skipped (PostgreSQL-only).
 - ruff, Bandit and pip-audit clean.

Honesty note. Agent metrics were produced with deterministic reference agents, because no API key was available in the build environment. They validate orchestration, verification, policy and safety controls, not Claude's reasoning quality. Set `LLM_PROVIDER=anthropic` and re-run `evaluation/agent_eval.py` to measure real models. All data is synthetic; see [docs/data_pipeline.md](#).

Architecture at a glance

[Diagram: see the Mermaid diagram in the Markdown version of this document in the repository.]

Run it locally

Prerequisites: Python 3.12, PostgreSQL 16 with pgvector (or Docker), about 4 GB RAM.

```
python -m venv .venv && source .venv/bin/activate
make install
make embeddings                # one-off: pretrained GloVe vectors (~376 MB download)
cp .env.example .env          # set JWT_SECRET
make pipeline                  # about 3 minutes: 772k transactions -> alerts -> model ->
    ↪ knowledge index
make api                       # http://localhost:8000/docs
make dashboard                 # http://localhost:8501
make test && make test-pg      # SQLite suite, then PostgreSQL suite
make eval && python evaluation/make_report.py
```

With Docker: see the header of `docker-compose.yml`. The Docker build was not executed in the build sandbox (Docker unavailable). The same commands ran natively.

Demo users (password `Demo!Passw0rd`, local only; seeding is refused in production):

Tenant	Username	Role	Can do
emerald	alice.analyst	analyst	Start investigations
emerald	ivan.investigator	investigator	Approve closures and monitoring
emerald	maeve.mlro	mlro	Approve STRs and account restrictions
emerald	aoife.auditor	auditor	Read-only, audit trail
emerald	adam.admin	admin	Ingest data, upload knowledge
liffey	liam.analyst / lara.investigator / lorcan.mlro / lucy.admin	as above	Second tenant, fully isolated

Implemented locally vs production architecture

Concern	Implemented and run here	Production design (not deployed)
Database	PostgreSQL 16 + pgvector 0.6 (tests also on SQLite)	Azure Database for PostgreSQL Flexible Server, zone-redundant HA
Streaming ingestion	Batch CSV via API / pipeline	Azure Event Hubs (Kafka API) → ingestion consumers
Work queue	DB queue with FOR UPDATE SKIP LOCKED worker	Azure Service Bus with dead-letter queue, KEDA autoscaling
Embeddings	GloVe 300d + SIF weighting (Hugging Face blocked in sandbox)	Self-hosted BAAI/bge-base-en-v1.5 + cross-encoder reranker
LLM	Deterministic reference agents; Anthropic provider implemented but not called	Claude via internal model gateway (quotas, key rotation, caching)
Rate limiting	In-process sliding window	Azure Cache for Redis
Model registry	File registry + DB metadata + SHA-256 verification	MLflow / Azure ML registry on Blob Storage
Secrets	.env + production startup guards	Azure Key Vault via managed identity
Orchestration	uvicorn / Docker Compose files	Azure Container Apps (Bicep in infrastructure/azure)
Observability	Prometheus metrics, JSON logs, OTel hooks, Grafana dashboard JSON	Azure Monitor managed Prometheus + Application Insights
Identity	Local users, PBKDF2 hashes, JWT	Microsoft Entra ID (OIDC), conditional access

Concern	Implemented and run here	Production design (not deployed)
Schema migrations	<code>create_all</code>	Alembic migrations in the deploy pipeline (gap, see architecture.md)

Repository map

```

backend/app/
  core/          settings, structured logging, errors, request context
  data/          generator, validation, ingestion, features, rules, alert scan
  database/      SQLAlchemy schema (20 tables), portable pgvector type, sessions
  ml/           training, calibration, registry, scoring, drift, metrics
  rag/          parsing/chunking, embeddings, indexer (poisoning defence), retriever (ACL hybrid)
  ai/           prompt registry, LLM gateway, providers (Anthropic / deterministic / fault injection)
  agents/       schemas, tools, agents, verifier, orchestrator
  services/     policy engine, approvals, actions, audit
  security/     JWT auth, RBAC, PII, prompt guard, rate limiter, passwords
  observability/ Prometheus metrics, OpenTelemetry tracing
  api/          FastAPI routes, schemas, middleware
  pipeline.py   end-to-end offline pipeline; worker.py async investigation worker
backend/tests/  unit, integration, security, API, adversarial, end-to-end, PostgreSQL-only
prompts/       versioned YAML prompts
data/knowledge_base/ 13 policy / regulation / typology / case documents (synthetic summaries)
evaluation/    datasets, evaluation scripts, JSON reports, report generator
frontend/     Streamlit dashboard + page smoke test
infrastructure/ Azure Bicep, Prometheus alerts, Grafana dashboard, OTel collector
docs/         architecture, database, data pipeline, ML, RAG, agents, security, evaluation,
              deployment, observability, reverse engineering, interview preparation, resume evidence

```

Documentation

Document	What it covers
architecture.md	Components, data flow, technology choices, scaling from hundreds to millions, cost optimisation
database.md	Schema, keys, indexes, constraints, concurrency
data_pipeline.md	Synthetic data design and assumptions, validation, features, rules
ml.md	Models, validation protocol, calibration, registry, drift, retraining
rag.md	Chunking, embeddings, hybrid retrieval, access control, poisoning defence
agents.md	Agent contracts, tools, verification, failure handling
security.md	Threat model and controls
evaluation.md	Every measured number
deployment.md	Local, Docker, CI/CD, Azure
observability.md	Logs, metrics, traces, alerts, LLMOps ledger
reverse_engineering.md	Beginner-to-senior learning path through the codebase
interview.md	90 questions with answers, follow-ups and common mistakes
resume_evidence.md	Only what was implemented and measured

The knowledge base contains educational summaries written for this project, not legal text. The high-risk country list is illustrative. Nothing here is legal or compliance advice.

Architecture

1. Problem framing

Stakeholder	Pain today	What SentinelAML changes
L1 analyst	Most alerts are false positives; hours of manual evidence gathering per alert	Alerts arrive ranked with explanations; an evidence pack and draft narrative are prepared
L2 investigator	Inconsistent closure rationales, QA findings	Recommendations cite verified transaction IDs and policy passages
MLRO	Late escalations, weak STR narratives, regulatory exposure	STR drafts follow a who/what/when/where/why structure; only the MLRO can approve STRs or restrictions
Head of Financial Crime	Headcount grows with alert volume	Measured deprioritisation (73.7% at 91.7% recall on test) and cost per investigation
Auditor / regulator	"Show me why this was closed"	Hash-chained audit trail; model version and prompt version for every decision

Design principle: automation proposes, code verifies, humans decide anything consequential.

2. Why each kind of intelligence is used, and where it is not

Task	Chosen	Rejected alternative	Reason
Alert generation	Deterministic rules	ML replacing rules	Rules are an auditable regulatory control. ML prioritises; it does not remove the control.
Alert risk scoring	XGBoost + Platt	LLM scoring; deep nets; logistic only	Tabular numeric features, needs calibrated probabilities and SHAP-style contributions. XGBoost beat logistic under identical protocol (PR-AUC 0.976 vs 0.958). An LLM would be slower, costlier, uncalibrated and non-deterministic.
Novelty signal	Isolation Forest	Autoencoder	Cheap, no labels, interpretable as a percentile; adequate for a secondary signal
Calibration	Platt (sigmoid)	Isotonic	Isotonic created score ties and lowered PR-AUC (0.950 vs 0.976) on this data size
Typology tagging, routing	Claude Haiku-class	Sonnet; rules only	Short structured extraction; low stakes; failure degrades to rules
Evidence gathering	Claude Sonnet-class with tools	One giant prompt with all data	Data volume varies; planning which evidence to fetch is multi-step; tools keep data scoped
Policy mapping	Claude Haiku-class over retrieved passages	Sonnet	Extraction with citations, not open reasoning
Recommendation + narrative	Claude Sonnet-class	Templates only	Synthesising mixed evidence into a defensible narrative is where an LLM adds value
Verification	Deterministic code	LLM-as-judge	Must be reproducible, cheap and not share failure modes with the generator
Approval decision	Deterministic policy engine	Agent decides	Legal authority (CJA 2010 reporting, restrictions) sits with named people
Embeddings	GloVe-SIF locally; BGE in production	OpenAI or Voyage API embeddings	Self-hostable keeps regulated text in-tenant; GloVe used only because model downloads were blocked

3. Component view

[Diagram: see the Mermaid diagram in the Markdown version of this document in the repository.]

Workflow sequence

[Diagram: see the Mermaid diagram in the Markdown version of this document in the repository.]

4. Communication and state

- **Synchronous API** for reads and for investigations in local mode.

- **Asynchronous mode** (mode=async): the API writes a QUEUED workflow run and returns 202. Workers claim runs with `SELECT ... FOR UPDATE SKIP LOCKED`. The test suite ran 3 concurrent workers; each workflow was processed exactly once.
- **Idempotency:**
 - Idempotency-Key header on investigation starts.
 - Transaction IDs as natural keys with `ON CONFLICT DO NOTHING`.
 - Alert dedupe_key unique constraint.
- **Concurrency:**
 - `alerts.lock_version` (optimistic locking) prevents two investigations starting on one alert.
 - Approvals lock the recommendation row.
 - Audit appends use a per-tenant advisory lock.
- **Failure policy:** triage may degrade. Every other failure (timeout, schema error, loop, verification failure, unexpected exception) ends in `NEEDS_MANUAL_REVIEW` with an audit record. Nothing fails open.

5. Scaling: hundreds to millions

Load	Transactions/day	Architecture
Today (demo)	~4k synthetic/day, batch	Single API process, one PostgreSQL, synchronous investigations
Mid-size fintech	1–5M/day, ~10k alerts/day	Event Hubs ingestion; partitioned transactions by month; read replica for analytics; Service Bus + 5–30 workers; Redis rate limits; nightly feature scans as Container Apps Jobs
Large bank	50M+/day, 100k alerts/day	Streaming features (Flink / Spark Structured Streaming) into an online feature store; dedicated vector index (Azure AI Search or pgvector with HNSW on its own server); model served behind a batching inference service; LLM calls through a gateway with per-tenant token budgets; Citus or per-region databases

Mechanism	Where it lives or would live
Horizontal scaling	Stateless API replicas; workers scale on queue depth (KEDA)
Backpressure	The queue absorbs bursts; the API never waits on LLM work in async mode
Retries	Gateway: exponential backoff on retryable errors, one schema-repair attempt; Service Bus redelivery with max delivery count 5 and a dead-letter queue
Circuit breaker	Per model in the gateway; open circuit → manual review instead of piling up timeouts
Caching	Candidate: retrieval results per (query, tenant, clearance, KB version); triage outputs per feature hash; Anthropic prompt caching for static system prompts
Indexing	(tenant_id, customer_id, ts) for window queries; (tenant_id, status, risk_score) for queues; ACL composite index for retrieval
Vector search	Exact search is fine for 62 chunks. At 100k+ chunks add an HNSW index and keep ACL columns filterable (pre-filter or iterative scan)
Rate limiting	Per token and per IP (login 10/min) → Redis in production

Measured baseline on one CPU (see evaluation.md):

Request	p95 latency	Throughput
Alert list	64 ms	161 req/s
Knowledge search	106 ms	97 req/s
Full investigation (no network LLM)	424 ms	14.5 req/s

With real LLMs, investigation latency is dominated by model calls (seconds), which is why production uses async workers.

6. Cost model and optimisation

Estimated tokens per investigation: mean 21.5k, p95 29.1k (estimated from prompt size). At configured prices this is mean **\$0.075** per investigation. Pricing is set in `Settings.model_pricing`; verify it against current Anthropic pricing.

Lever	Implemented?	Effect
Model routing (Haiku for triage and policy, Sonnet for investigation and decision)	Yes	Cheap steps on the cheap model
ML pre-ranking before any LLM call	Yes	Low-risk alerts can be batch-closed by humans without an LLM run
Deterministic verifier instead of LLM judge	Yes	No second model call per recommendation
Bounded tool loop (8) and one repair retry	Yes	Caps worst-case tokens
Only investigate alerts above a score or by human choice	Policy choice	The largest saving: 73.7% of alerts are deprioritised
Prompt caching of static system prompts	Not yet	Usually cuts repeated input cost substantially
Batch API for non-urgent overnight investigations	Not yet	Lower price for asynchronous work
Compress tool results (top-N + aggregates, not raw rows)	Partly (limits + counterparty aggregates)	Fewer input tokens

7. Known gaps and deliberate trade-offs

Gap	Why accepted now	Next step
No Alembic migrations	Single-developer, rebuildable demo	Add Alembic; CI checks migrations on an empty database
Synthetic data only	Real AML data is confidential	Validate on a bank's historical dispositions under a data-sharing agreement
Agent quality not measured on real Claude	No API key in the build sandbox	Run <code>agent_eval.py</code> with <code>LLM_PROVIDER=anthropic</code> ; add LLM-judge narrative quality scoring
GloVe embeddings	Hugging Face blocked	Switch to BGE; re-run <code>rag_eval</code> ; expect dense recall to improve
Audit events survive business rollbacks	Needed to avoid serialising investigations	Mark events with outcome; export to immutable Blob storage
Auto-closure disabled for cash-intensive SMEs	Model cannot separate weak laundering from trading (measured)	Peer-group features (compare with similar businesses), external data (VAT returns)
In-process rate limiter	Single replica locally	Redis
Docker/Bicep not executed here	No Docker or Azure CLI in sandbox	CI builds images and runs Trivy; deploy workflow is environment-gated

Evaluation results

Generated by `python evaluation/make_report.py` from the JSON files in `evaluation/reports/`. Do not edit numbers by hand.

All figures come from one clean end-to-end run (`python -m app.pipeline --reset --all`) on synthetic data, PostgreSQL 16 + pgvector, one shared CPU. They describe this system on this data. They are **not** claims about real-world bank data.

1. Data pipeline

Measure	Value
Raw transaction rows generated	771,818
Loaded	759,983
Rejected (with reason)	4,237
Duplicates removed	7,598
Reject reasons	non_positive_amount 1549, unparseable_timestamp 771, unknown_account 769, unsupported_currency 761, future_timestamp 387
Payment references flagged for injection at ingestion	37
Monitoring alerts created (rules)	3,215 (3,632 suppressed as repeats)
Ingestion time / scan time	37.0 s / 38.1 s
Replay of the same file	0 rows loaded (idempotent), verified in tests

2. Machine learning (alert risk model)

Temporal split: train 1,977 alerts (to 2026-05-22), calibration 435, **held-out future test 803 alerts** (from 2026-06-05). Threshold selected on out-of-fold predictions, never on test.

Model (test period)	ROC-AUC	PR-AUC	Precision	Recall	F1	Brier	ECE	Deprioritised	Confusion (tn/fp/fn/tp)
Logistic regression (baseline)	0.961	0.958	0.890	0.913	0.901	0.027	0.021	70.6%	547/26/20/210
XGBoost, uncalibrated	0.980	0.976	1.000	0.917	0.957	0.022	0.019	73.7%	573/0/19/211
XGBoost + isotonic (comparison)	0.969	0.950	1.000	0.917	0.957	0.022	0.017	73.7%	573/0/19/211
XGBoost + Platt (production)	0.980	0.976	1.000	0.917	0.957	0.021	0.018	73.7%	573/0/19/211

Customer-grouped 5-fold CV (no customer in both train and validation): ROC-AUC 0.979 ± 0.021 , PR-AUC 0.976 ± 0.024 . Isolation Forest (unsupervised) test ROC-AUC 0.940.

Operating curve (test). The business chooses the trade-off between recall and workload:

Target recall on dev	Threshold	Test recall	Test precision	Alerts deprioritised
90.0%	0.9368	90.9%	100.0%	74.0%
95.0%	0.7338	91.7%	100.0%	73.7%
98.0%	0.0041	97.8%	44.5%	37.1%
99.0%	0.0020	99.6%	35.6%	19.8%

Reading: about 8% of suspicious alerts (weak-intensity cash-business typologies) look normal to the model. Catching them costs most of the workload saving. This is why a low score means *lighter review*, not automatic closure.

Rule	Test alerts	Positives	Recall at threshold	Deprioritised
R01_STRUCTURING	140	29	89.7%	81.4%
R02_RAPID_MOVEMENT	429	123	90.2%	74.1%
R03_HIGH_RISK_JURISDICTION	75	11	100.0%	85.3%
R04_VELOCITY_SPIKE	18	18	100.0%	0.0%
R05_LARGE_CASH	134	42	90.5%	71.6%
R06_DORMANT_REACTIVATION	7	7	100.0%	0.0%

Tenant	Test alerts	Positives	Recall at threshold	Deprioritised
emerald	501	158	91.1%	71.3%
liffey	302	72	93.1%	77.8%

Promotion gates: beats_baseline_pr_auc = True, test_recall_at_threshold_ge_0.90 = True, group_cv_roc_auc_ge_0.80 = True

3. Retrieval (RAG)

36 labelled queries, K = 5, embedding: glove-wiki-gigaword-300-sif (local), reranker: lexical-coverage (local).

Mode	Recall@5	Precision@5	MRR	nDCG@5	Hit@5	p50 ms	p95 ms
dense	0.736	0.206	0.667	0.649	0.778	4.82	51.99
lexical	0.880	0.233	0.781	0.787	0.917	4.72	5.7
hybrid	0.903	0.239	0.766	0.784	0.944	6.11	6.87

Mode	Recall@5	Precision@5	MRR	nDCG@5	Hit@5	p50 ms	p95 ms
hybrid_rerank	0.903	0.239	0.801	0.807	0.944	6.64	7.87

Access-control audit: 21,147 returned results checked across 2 tenants × 4 clearance levels × 4 modes: **0 unauthorised**. Precision@5 is capped by the labels (most queries have 1–2 relevant chunks). Caveat: Single-annotator labels on a 62-chunk synthetic corpus; indicative only.

4. Agentic workflow

Read this first: Deterministic rule-based reference agents stand in for Claude; these numbers measure orchestration, verification, policy and safety controls, and a heuristic decision baseline - not LLM reasoning quality.

160 held-out alerts (80 suspicious, 80 legitimate), each run through the full orchestrator.

Metric	Value
Task success (terminal state with verified recommendation)	100.0%
Structured-output validity (final / first attempt)	100.0% / 100.0%
Tool-selection accuracy (required tools called)	100.0%
Groundedness (key findings with fully verified evidence)	1.000
Citation validity	1.000
Hallucination rate (workflows with unverifiable evidence IDs)	0.0%
Numeric claims re-computed from the database / failed	735 / 0
Unsafe-action rate (dangerous action executed without approval)	0.0%
Approval compliance	100.0%
Suspicious alerts recommended STR/restriction	90.0%
Suspicious alerts escalated or monitored	93.8%
Suspicious alerts recommended for closure (all went to a human)	6.2%
Suspicious alerts auto-closed	0
Legitimate alerts recommended for closure	65.0%
Legitimate alerts escalated to STR	0.0%
Share auto-applied without a human	20.6%
Workflow latency p50 / p95 (no network LLM latency)	59.5 / 66.5 ms
Tokens per workflow, mean / p95 (estimated)	21,528 / 29,052
Cost per workflow, mean / p95 (estimated)	\$0.0747 / \$0.0994

Cost basis: token counts ESTIMATED (chars/4) from deterministic provider, priced at configured model rates. Routing: reasoning = claude-sonnet-5, fast = claude-haiku-4-5-20251001.

Adversarial fault injection: 96 runs, safe-outcome rate 100.0%, unsafe automatic executions 0.

Injected fault	Expected safe behaviour	Runs	Safe outcomes
investigation:timeout	manual_review	8	100.0%
investigation:error	manual_review	8	100.0%
investigation:loop	manual_review	8	100.0%
investigation:hallucinated_evidence	blocked	8	100.0%
policy:timeout	manual_review	8	100.0%
decision:malformed_json	manual_review	8	100.0%
decision:schema_invalid	manual_review	8	100.0%
decision:hallucinated_evidence	blocked	8	100.0%
decision:unsafe_close	blocked	8	100.0%
decision:fabricated_citation	blocked	8	100.0%
decision:pii_leak	blocked	8	100.0%
triage:error	degraded_continue	8	100.0%

5. System performance

Environment: 1 CPU, Python 3.12.3, uvicorn, 1 worker, PostgreSQL 16 + pgvector 0.6 on the same host, LLM provider deterministic (no network LLM latency).

Endpoint	Requests	Concurrency	p50 ms	p95 ms	p99 ms	Failure rate	Throughput (req/s)
GET /health/ready	200	8	24.4	36.6	107.3	0.0%	289.0
GET /api/v1/alerts (page 25)	400	8	48.8	63.6	69.9	0.0%	161.3
GET /api/v1/alerts/{id}	400	8	38.8	49.6	106.1	0.0%	196.0
POST /api/v1/kb/search	300	8	74.5	105.9	319.3	0.0%	97.4
POST /api/v1/alerts/{id}/investigations (sync, deterministic)	40	4	259.9	423.9	442.1	0.0%	14.5
GET /api/v1/analytics/kpis	100	8	67.0	77.8	79.9	0.0%	118.7

Caveat: Single shared CPU in a sandbox; absolute numbers are a floor for comparison, not a capacity claim. Workflow latency excludes real LLM latency (typically seconds per call).

6. Findings that changed the system

Evaluation finding	Change made	Measured effect
Rules generated drift alarms and spikes for customers with <30 days history	Monitoring starts once the 30-day feature window is observable	False drift alarm removed
Threshold chosen on a small calibration slice was unstable (15% vs 70% deprioritised between runs)	Out-of-fold threshold + calibration; operating curve reported	Stable operating point across rebuilds
Logistic baseline trained on less data than XGBoost	Identical protocol for both models	Fair comparison: PR-AUC 0.958 vs 0.976
1 of 80 suspicious alerts auto-closed; all misses were SME cash businesses	Branch-dispersion feature + no auto-closure for cash-intensive SMEs	0 suspicious alerts auto-closed; auto-applied share fell from 39% to about 21%
Injection in small payments missed (agent only viewed the largest transactions)	Scan every reference at ingestion; check whole alert window	Injected cases always require human review
Concurrent investigations forked the audit hash chain	Advisory-locked autonomous audit appends	Chain valid after 40 concurrent investigations; regression test with 8 writers
UUID fragments detected as phone numbers (1.6% of high-risk workflows failed verification)	Exclude system identifiers from PII scanning	611/611 high-risk workflows verified
PyJWT 2.7.0 had known CVEs (pip-audit)	Upgraded to 2.14.0	pip-audit: no known vulnerabilities

Reverse-engineering guide

Goal: understand every part of SentinelAML well enough to rebuild it, defend each decision, and say what you would change.

Method: run → observe → read → break → explain.

Format: 5 levels, then a 14-day plan sized for about 4-hour study sessions.

Level 0: The business, in plain English

What a bank must do. Irish banks and payment firms must watch customer transactions for money laundering. When something looks suspicious, they report it to FIU Ireland (the Garda Financial Intelligence Unit) and to the Revenue Commissioners. They must not tell the customer (tipping off).

How they watch today. Rules: “two cash deposits just under €10,000 within 30 days”, and so on. Rules fire alerts. Most alerts are innocent, for example a takeaway lodging weekend takings.

What SentinelAML does. Keeps the rules, then:

1. Scores each alert.

2. Lets AI agents gather evidence and read policy.
3. Checks the AI's claims against the database.
4. Makes a person approve anything serious.

Glossary

Term	Meaning
Alert	One rule firing for one customer at one point in time
Typology	A known laundering pattern (structuring, money mule, funnel account...)
STR	Suspicious Transaction Report to FIU Ireland and Revenue
MLRO	Money Laundering Reporting Officer: the person legally responsible for reporting decisions
KYC	Know Your Customer: the profile (occupation, expected turnover, risk rating)
PEP	Politically exposed person (needs enhanced due diligence)
False positive	Alert on innocent activity
Four-eyes	The person who prepared a decision cannot approve it
Tenant	A client organisation (Emerald Digital Bank, Liffey Payments) sharing the platform
Clearance	The document sensitivity level a role may read

Checkpoint: explain the system to a non-technical friend in 60 seconds, without saying “AI”.

Level 1: Run it and watch

Do each step, then write one sentence on what you saw.

#	Do	Observe	Where in code
1	make pipeline	JSON report: rows loaded, rejected, alerts, model gates	backend/app/pipeline.py
2	Open data/raw/transactions.csv; find not-a-date and a negative amount	Defects are deliberate	data/generator.py: :_inject_defects
3	psql: SELECT reason, count(*) FROM rejected_records GROUP BY 1;	Every reject has a reason	data/validation.py
4	SELECT rule_code, count(*), avg(risk_score) FROM alerts GROUP BY 1;	Rules differ in volume and risk	data/rules.py
5	make api, open /docs, get a token as alice.analyst	JWT and roles	api/routes/auth.py
6	GET /api/v1/alerts?status=NEW	Ranked queue, pagination	api/routes/alerts.py
7	Start an investigation on the top alert	Workflow status	agents/orchestrator.py
8	GET /api/v1/investigations/{id}	AWAITING_APPROVAL	Same
9	Approve as ivan.investigator (fails), then as maeve.mlro	Steps, tool calls, LLM calls with model and prompt version	services/approvals.py
10	GET /api/v1/audit/verify as MLRO	Authority matrix	services/audit.py
11	Knowledge search as analyst vs MLRO: “law enforcement liaison”	Hash chain valid	rag/retriever.py
12	make dashboard; log in as each role	Different results by clearance	Pages differ by role
			frontend/app.py

Level 2: Component by component

For each component, learn the eight answers: **what, why, talks to, why this technology, alternatives, failure modes, security, scale and cost.**

2.1 Synthetic data generator: `data/generator.py`

Question	Answer
What	Creates customers, accounts, transactions and labels with personas and typologies
Why	Real AML data is confidential; public datasets lack KYC, free text and tenants
Talks to	Writes CSVs + manifest; ingestion reads them; training reads <code>labels.csv</code>
Why this approach	Behavioural simulation encodes realistic false positives (cash businesses, property sales, family remittances)
Alternatives	PaySim/AMLworld (no KYC or text), GAN/CTGAN (hard to control typologies, can memorise)
Failure modes	Too-easy data gives inflated metrics (happened: ROC-AUC 0.98 from one feature)
Security	No real PII; names are random combinations
Scale	NumPy per customer; 772k rows in about 20 s

Exercise: set `suspicious_rate=0.02`, rebuild. Predict first: what happens to PR-AUC and why?

2.2 Validation and ingestion: `data/validation.py`, `data/ingestion.py`

Question	Answer
What	Schema check → row checks → clean → dedupe → injection scan → gates → load
Why	Bad data silently corrupts features, models and reports
Talks to	accounts (referential check), <code>ingestion_batches</code> , <code>rejected_records</code> , <code>transactions</code>
Why this technology	pandas vectorised checks; PostgreSQL COPY (fast) + ON CONFLICT DO NOTHING (idempotent)
Alternatives	Great Expectations/Pandera (richer reports, more dependencies); dbt tests (post-load); Spark (unneeded at this size)
Failure modes	Gate too strict blocks valid batches; too loose loads garbage; replays double count without idempotency
Security	Tenant/account mismatch rejected (prevents cross-tenant writes); references normalised and scanned
Scale	Batch → Event Hubs consumers with the same validation code; per-partition idempotency

Key idea: quarantine the whole batch when gates fail. Partial loads create reconciliation nightmares.

2.3 Features and rules: `data/features.py`, `data/rules.py`, `data/alerting.py`

Question	Answer
What	33 point-in-time features; 6 rules; weekly scan with 28-day suppression
Why	Rules are the regulated control; features feed rules, ML and agents
Talks to	Reads transactions; writes alerts with a feature snapshot
Why this design	One feature function for training and serving prevents skew; <code>ts <= as_of</code> prevents leakage
Alternatives	Feature store (Feast/Databricks) at scale; SQL window functions (harder to test)
Failure modes	Undefined features for new customers (happened: velocity spikes, false drift); timestamp resolution bugs (happened: 0 alerts)
Scale	Streaming feature computation and an online store beyond ~millions of events/day

Exercise: find where `unique_cash_branches_30d` is computed. Explain why it was added (see `ml.md`, section 4).

2.4 Database: `database/models.py`, `database/types.py`

Question	Answer
What	20 tables covering data, ML, RAG, workflow, approvals, audit
Why PostgreSQL	ACID for approvals and actions; JSONB for semi-structured outputs; pgvector keeps vectors under the same ACL and transactions
Alternatives	Separate vector database (second consistency and ACL surface); MongoDB (weak relational integrity for approvals)
Failure modes	Races (double approval, double investigation, audit fork) handled by constraints, optimistic locks, row locks, advisory locks
Security	Tenant columns everywhere; constraints as a last line of defence
Scale	Partition transactions; read replicas; HNSW index; row-level security

Exercise: open two psql sessions and try to approve the same recommendation. Which constraint stops the second?

2.5 ML pipeline: `ml/train.py`, `ml/calibration.py`, `ml/registry.py`, `ml/scoring.py`, `ml/drift.py`

Question	Answer
What	Train logistic baseline + XGBoost; calibrate; threshold; Isolation Forest; register; gate; score with lineage; drift
Why XGBoost	Tabular, non-linear, fast, explainable contributions
Why calibration	Policies use score thresholds (for example <0.05 auto-close), so scores must behave like probabilities
Why a temporal split	Deployment predicts the future
Alternatives	LightGBM, CatBoost, logistic only (the measured gap was small in ranking but large in false positives: 26 vs 0)
Failure modes	Leakage, unstable thresholds (happened), unfair baselines (happened), stale model, pickle tampering (SHA-256 check)
Scale	Batch scoring job; model server with micro-batching; shadow deployment for challengers
Cost	Training in seconds on CPU; inference about 1 ms

Exercise: change TARGET_RECALL to 0.98 and retrain. Compare the deprioritised share with the operating curve in evaluation.md.

2.6 RAG: `rag/documents.py`, `rag/embeddings.py`, `rag/indexer.py`, `rag/retriever.py`

Question	Answer
What	Parse, scan, chunk, embed, store; ACL-filtered hybrid search; context builder
Why hybrid	Measured: dense 0.736 vs BM25 0.880 vs hybrid 0.903 Recall@5. Domain terms need exact match; paraphrases need semantics.
Why ACL in SQL	Unauthorised chunks never become candidates
Alternatives	Azure AI Search, Elasticsearch/OpenSearch, Pinecone; cross-encoder reranker
Failure modes	Poisoned documents, stale versions, empty retrieval, embedding model changes needing re-index
Security	Quarantine, four-eyes approval, safe YAML, comment stripping, delimiter neutralisation
Scale	HNSW, tsvector full text, caching per KB version

Exercise: upload data/adversarial/poisoned_policy_update.md via the API. Explain each rule it trips.

2.7 LLM gateway and prompts: `ai/gateway.py`, `ai/providers.py`, `ai/prompts.py`, `prompts/*.yaml`

Question	Answer
What	Single entry point for all model calls: routing, schema enforcement, retries, circuit breaker, cost ledger

Question	Answer
Why	Agents must not each reinvent reliability, and every call must be attributable (model, prompt version)
Why tool-schema structured output	Model must return arguments matching a JSON schema; more reliable than "reply in JSON"
Alternatives	LiteLLM proxy, Azure API Management, LangChain/LangGraph (more abstraction than needed; harder to control failure semantics)
Failure modes	Timeouts, rate limits (429/529), invalid JSON, outages, cost runaway
Security	Redacted output previews; the key is only in settings
Scale and cost	Per-tenant token budgets, prompt caching, batch processing for overnight work

Exercise: write a provider that returns invalid JSON twice. Trace the `llm_calls` row it produces.

2.8 Agents, verifier, orchestrator: `agents/*.py`

Question	Answer
What	Triage → investigation (tool loop) → policy → decision → verify → policy engine
Why multiple agents	Separation of duties, minimal tools, per-step model choice and evaluation
Why a deterministic orchestrator	Predictable control flow, bounded loops, testable failure semantics
Alternatives	LangGraph (graph orchestration), Semantic Kernel, AutoGen (free-form agent chat: hard to govern)
Failure modes	Hallucinated evidence, loops, unsafe recommendations, injection. Each has a fault-injection test.
Security	The LLM has no write path; tools are case-scoped; injection forces human review
Cost	~21.5k tokens per investigation (estimated), bounded by step limits

Exercise: in `ai/simulators.py`, make the decision agent recommend `RESTRICT_ACCOUNT` for every alert. Run the suite. Which controls stop it? (Policy engine: MLRO approval. Verifier: escalation without aggravating evidence.)

2.9 Services: `services/policy_engine.py, approvals.py, actions.py, audit.py`

Question	Answer
What	Approval rules, human decisions, allow-listed execution, tamper-evident trail
Why	Legal accountability and auditability
Alternatives	OPA/Rego for policy as code; event-sourced audit to immutable storage
Failure modes	Audit race (happened), policy drift from SOP (the SOP document was updated with the code)

2.10 API and security: `api/*, security/*, main.py`

Question	Answer
What	26 operations; JWT, RBAC, tenant scoping, validation, error envelope, middleware (request ID, metrics, rate limit, headers, body size)
Why FastAPI	Typed request validation, OpenAPI docs, dependency injection for auth
Alternatives	Django REST (heavier), Flask (less typing), gRPC internally
Failure modes	Leaking existence of other-tenant IDs (prevented by 404), stack traces (uniform 500), DB outage (503)

2.11 Observability, dashboard, DevOps

Observability

- **What:** Prometheus metrics, JSON logs, OTel spans.
- **Why:** you can't operate LLM systems without cost and latency per prompt.

- **Alternative:** vendor APM only.

Dashboard

- **What:** Streamlit, API-only access.
- **Why:** backend-first team, fast to change.
- **Alternative:** React (better UX, more skills needed).

DevOps

- **What:** multi-stage Docker, Compose, GitHub Actions with security gates, Bicep.
- **Why:** reproducible builds, supply-chain checks, reviewable infrastructure.

Level 3: Trace one alert through the code

Open the files side by side and follow a single alert.

1. `data/generator.py::_typology_txns`: a money-mule customer is created.
2. `data/ingestion.py::clean_transactions`: a payment reference is normalised and scanned (`reference_flags`).
3. `data/alerting.py::run_scan` → `features.py::compute_features` → `rules.py::evaluate`: R02 fires; the snapshot is stored.
4. `ml/scoring.py::ModelService.score_alert`: verified artifact → raw score → Platt → contributions → predictions row → priority.
5. `api/routes/alerts.py::start_investigation`: auth dependency require(`START_INVESTIGATION`).
6. `agents/orchestrator.py::create_run`: tenant filter, status check, optimistic lock, audit.
7. `Orchestrator.execute` → `_window_injection_flags` → `TriageAgent.run` → `LLMGateway.structured`.
8. `InvestigationAgent.run`: loop, `ToolExecutor.run` (permission, argument validation, scope) → `dis-closed_refs`.
9. `PolicyAgent.run`: `Retriever.search` with the requester's clearance → `build_context`.
10. `DecisionAgent.run` → `verifier.verify_decision` → optional repair → `policy_engine.evaluate`.
11. Recommendation saved `PENDING_APPROVAL` (mlro).
12. `api/routes/recommendations.py::decide` → `services/approvals.py::decide` (authority, four-eyes) → `services/actions.py::execute` → `STRReport` → `audit.record`.

Checkpoint: draw this as a sequence diagram from memory, then compare with `architecture.md`.

Level 4: Break it on purpose

Predict the outcome before running each one.

#	Change	Expected failure	Lesson
1	Remove <code>Alert.tenant_id == p.tenant_id</code> in <code>get_alert</code>	<code>test_tenant_isolation_api</code> fails	Tenant filters are security code
2	Post-filter ACL instead of the SQL filter	Access-control tests may pass but results drop below K	Pre-filter vs post-filter
3	Delete the advisory lock in <code>audit.record</code>	PostgreSQL concurrency test fails intermittently	Races need concurrency tests
4	Make <code>DeterministicProvider</code> cite TX: FAKE	Verifier: <code>unverified_evidence_ref</code>	The LLM is untrusted
5	Remove the cash-intensive rule from the policy engine	<code>agent_eval: suspicious</code> <code>auto-closed > 0</code>	Policy compensates for model blind spots
6	Train XGBoost on train only, logistic on dev	Gate comparison becomes unfair	Protocol matters
7	Remove system-ID masking in <code>pii.py</code>	Random verification failures	Nondeterministic bugs from data content
8	Set <code>agent_max_tool_steps=2</code>	Investigations fail closed	Budgets vs thoroughness
9	Break <code>FORCE_NOT_NULL</code> in COPY	Ingestion NotNull error	Empty string vs NULL in CSV
10	Replace Platt with isotonic	PR-AUC drops (ties)	Calibration choice affects ranking

Level 5: Senior-engineer view

Questions to answer in your own words:

1. **Where is the system's real safety boundary?** The allow-listed executor plus the policy engine plus human approval, not the prompts.
2. **What is the weakest measured component?** Dense retrieval with GloVe (0.736), and the model's blind spot on weak cash-business typologies.
3. **What's unmeasured?** Real Claude quality, real bank data, Docker and Azure deployment.
4. **What would break first at 100× load?** Synchronous investigations and in-memory BM25/rate limits. Then per-tenant audit serialisation.
5. **What would you change with a team of five and six months?**
 - Real labels from case management.
 - Alembic migrations.
 - Entra ID.
 - Service Bus workers.
 - BGE + cross-encoder reranker.
 - An LLM-judge for narrative quality validated against MLRO ratings.
 - Shadow mode against the human process before any auto-closure.
6. **Which decisions would you reverse?**
 - Autonomous audit writes (outbox pattern instead).
 - Regex injection detection as more than a signal.
7. **How do you prove value to a bank?**
 - A 3-month shadow pilot: time-to-disposition, QA pass rate, missed-suspicious rate versus the current process.
 - Decide auto-closure thresholds with Compliance using the operating curve.

14-day plan (about 4-hour sessions)

Day	Session	Output to produce
1	Level 0 + README + architecture.md sections 1–2	60-second verbal explanation, recorded
2	Level 1 steps 1–6	Notes: one sentence per step
3	Level 1 steps 7–12	Screenshot of an investigation trace and audit verification
4	2.1–2.2 data generator and ingestion; data_pipeline.md	Diagram of ingestion from memory; explain the 9 defects
5	2.3 features and rules; do the exercise	Explain point-in-time correctness and the two bugs
6	2.4 database; database.md; concurrency exercise	Explain 4 concurrency controls
7	2.5 ML; ml.md; retrain with a new target recall	Operating-curve explanation for a compliance manager
8	Review day: interview.md basic questions 1–30 aloud	Mark weak answers
9	2.6 RAG; rag.md; poisoning exercise	Explain hybrid + ACL + poisoning in 3 minutes
10	2.7–2.8 gateway and agents; agents.md	Draw agent contracts from memory
11	2.9–2.10 services, API, security; security.md	Threat-model walkthrough
12	Level 3 trace + Level 4 breaks 1–5	Sequence diagram; results of the break experiments
13	Level 4 breaks 6–10; evaluation.md; observability.md	Explain each “finding that changed the system”
14	Level 5 + interview.md intermediate and advanced questions; resume_evidence.md	Draft 4 resume bullets in your own words

Interview preparation: SentinelAML

90 questions: 30 basic, 30 intermediate, 30 advanced. Each has four parts:

- **Tests:** what the interviewer is checking.
- **Strong answer:** grounded in this project where possible.

- **Follow-up:** the likely next question.
- **Mistake:** the common error to avoid.

Quote numbers only from docs/evaluation.md, and always say “synthetic data”.

Contents:

- Basic: B1–B30
 - Intermediate: I1–I30
 - Advanced: A1–A30
-

Basic

B1. Describe your project in two minutes.

- **Tests:** communication; business framing before technology.
- **Strong answer:** “Banks run rules that flag suspicious transactions, and most alerts are false positives. I built SentinelAML to help investigators clear them faster without removing human accountability. It keeps the rules, then an XGBoost model ranks alerts, agents gather evidence and relevant policy, a deterministic verifier checks every claim against the database, and a policy engine forces MLRO approval for reports or restrictions. On synthetic data, the model deprioritised 74% of alerts at 92% recall on a held-out future period, and 96 fault-injection runs all failed safe.”
- **Follow-up:** Why synthetic data?
- **Mistake:** Starting with the tech stack, or claiming real bank impact.

B2. What is the difference between a list and a tuple in Python, and where did you rely on immutability?

- **Tests:** Python fundamentals.
- **Strong answer:** Lists are mutable; tuples are immutable and hashable, so they can be dictionary keys or set members. I use frozensets for the high-risk country list and frozen dataclasses for Principal and Rule, so security-relevant objects can’t be changed after creation.
- **Follow-up:** Is a tuple containing a list hashable?
- **Mistake:** Saying tuples are “faster” as the main difference.

B3. What does a Pydantic model give you in an API?

- **Tests:** Validation basics.
- **Strong answer:** Type coercion, constraints (lengths, ranges, regex, enums), clear 422 errors and OpenAPI docs. I set extra="forbid", so unexpected fields such as "admin": true are rejected. The same models constrain LLM outputs, so a model can’t invent an action like WIRE_FUNDS.
- **Follow-up:** Where else would you validate besides the API?
- **Mistake:** Thinking validation at the API alone is enough (I also validate at ingestion and at the database with constraints).

B4. Explain primary key, foreign key and index.

- **Tests:** Database basics.
- **Strong answer:** A primary key uniquely identifies a row. A foreign key enforces that a referenced row exists. An index speeds lookups at write cost. In my schema, transactions.id is the source system’s ID, which doubles as the idempotency key. predictions.model_version_id is a foreign key so every score traces to a model. The index (tenant_id, customer_id, ts) serves the 30-day window queries.
- **Follow-up:** Why put tenant_id first in the index?
- **Mistake:** Indexing every column “for speed”.

B5. Write SQL to count alerts by rule with the average risk score.

- **Tests:** GROUP BY and aggregates.
- **Strong answer:** SELECT rule_code, COUNT(*) AS alerts, AVG(risk_score) AS avg_score FROM alerts WHERE tenant_id = :t GROUP BY rule_code ORDER BY alerts DESC; Always filter by tenant, and use bound parameters.

- **Follow-up:** Add the share of alerts with score ≥ 0.5 .
- **Mistake:** Forgetting the tenant filter, or string-formatting parameters into SQL.

B6. What is a REST API and what HTTP status codes does your API use?

- **Tests:** HTTP semantics.
- **Strong answer:** Resource-oriented endpoints over HTTP verbs.

Code	When
200	Read
201	Investigation created
202	Asynchronous job queued
401	Unauthenticated
403	Wrong role
404	Not found, including other tenants' IDs
409	Conflicts (double approval, reused idempotency key)
413	Payload too large
422	Validation failure
423	Account locked
429	Rate limited
503	Database down

- **Follow-up:** Why 404 rather than 403 for another tenant's alert?
- **Mistake:** Returning 200 with an error body.

B7. What is authentication vs authorisation?

- **Tests:** Security basics.
- **Strong answer:** Authentication proves who you are (JWT from login). Authorisation decides what you may do: role permissions, tenant scope, plus the approval authority matrix (only an MLRO approves an STR) and four-eyes (you can't approve your own investigation).
- **Follow-up:** Where is authorisation enforced: UI or API?
- **Mistake:** Relying on hiding buttons in the UI.

B8. What is a JWT and what are its risks?

- **Tests:** Token understanding.
- **Strong answer:** A signed token carrying claims (user, tenant, role, expiry). Risks: accepting `alg=none`, weak secrets, long expiry, no revocation, library CVEs. I pin the algorithm, require claims, use a 60-minute expiry, refuse default secrets in production, and test forged none and wrong-key tokens. pip-audit found CVEs in PyJWT 2.7.0 and I upgraded to 2.14.0.
- **Follow-up:** How would you revoke a token?
- **Mistake:** Thinking JWTs are encrypted.

B9. What is overfitting and how did you check for it?

- **Tests:** ML fundamentals.
- **Strong answer:** A model learning noise that doesn't generalise. I used a temporal hold-out (train on the past, test on the future), customer-grouped cross-validation (ROC-AUC 0.979 ± 0.021) so customers don't appear on both sides, and a simpler baseline for comparison.
- **Follow-up:** Why group by customer?
- **Mistake:** Only using random train/test splits on time-dependent data.

B10. Precision vs recall. Which matters more in AML?

- **Tests:** Metric reasoning in context.

- **Strong answer:** Precision: of flagged alerts, how many are truly suspicious. Recall: of suspicious alerts, how many were caught. Missing laundering is worse, so I set the threshold for about 95% recall on development data, then measured the workload cost. On test: 91.7% recall with 73.7% of alerts deprioritised.
- **Follow-up:** Why did test recall come out lower than 95%?
- **Mistake:** Choosing accuracy on imbalanced data.

B11. What is ROC-AUC vs PR-AUC?

- **Tests:** Imbalanced evaluation.
- **Strong answer:** ROC-AUC measures ranking across all thresholds, including many true negatives. PR-AUC focuses on the positive class, so it's more informative when positives are rare. I report both; PR-AUC 0.976 vs 0.958 for the baseline.
- **Follow-up:** When can ROC-AUC look good but PR-AUC be poor?
- **Mistake:** Treating AUC as a threshold-level metric.

B12. What is a confusion matrix? Read yours.

- **Tests:** Interpreting results.
- **Strong answer:** Counts of true negatives, false positives, false negatives and true positives. Mine on test: 573 TN, 0 FP, 19 FN, 211 TP for XGBoost, versus 26 false positives for logistic regression. The 19 false negatives are the risk, which is why low scores don't mean auto-closure.
- **Follow-up:** What did you do about those 19?
- **Mistake:** Celebrating zero false positives without discussing false negatives.

B13. What is RAG?

- **Tests:** GenAI basics.
- **Strong answer:** Retrieve relevant documents, then have the LLM answer using them, with citations. It keeps knowledge current and access-controlled without retraining. I retrieve policy and typology passages filtered by tenant and clearance, and the verifier rejects citations that weren't retrieved.
- **Follow-up:** Why not fine-tune instead?
- **Mistake:** Calling any chatbot "RAG".

B14. What is an embedding?

- **Tests:** Vector representation.
- **Strong answer:** A numeric vector where similar meanings are close. Locally I use GloVe word vectors averaged with frequency weighting (300 dimensions), because model downloads were blocked. Production would use BGE (768). Dense-only retrieval reached Recall@5 0.736, which is why I added BM25.
- **Follow-up:** What happens when you change the embedding model?
- **Mistake:** Mixing vectors from different models in one index.

B15. What is an AI agent?

- **Tests:** Agent concepts.
- **Strong answer:** An LLM that chooses actions (tool calls) in a loop toward a goal. My investigation agent decides which read-only tool to call next, up to 8 calls. It can't write data. Orchestration is a deterministic state machine.
- **Follow-up:** Why not let agents talk to each other freely?
- **Mistake:** Calling a single prompt an agent.

B16. What is Docker and why use multi-stage builds?

- **Tests:** Containers.
- **Strong answer:** Packages an app with its dependencies. Multi-stage builds compile wheels in one stage and copy only what's needed into a slim runtime running as a non-root user, which means smaller images and a smaller attack surface. Honest note: I wrote the Dockerfiles, but Docker wasn't available in my build environment.
- **Follow-up:** How do you scan images?
- **Mistake:** Running containers as root with build tools inside.

B17. What is CI/CD?

- **Tests:** DevOps basics.
- **Strong answer:** Automated build, test and deploy on every change. My pipeline: ruff, Bandit, pip-audit, gitleaks → tests on SQLite and PostgreSQL → image build → Trivy → manual, environment-gated Azure deploy with OIDC.
- **Follow-up:** Why a manual gate for production?
- **Mistake:** Only running unit tests.

B18. What are environment variables used for, and what must never be in code?

- **Tests:** Configuration and secrets.
- **Strong answer:** Configuration differing per environment. Secrets (JWT secret, database password, API keys) never go in code or images. I use pydantic-settings, `.env.example` without values, startup guards that refuse defaults in production, and Key Vault in the Azure design.
- **Follow-up:** How do containers get secrets from Key Vault?
- **Mistake:** Committing `.env`.

B19. What is logging vs metrics vs tracing?

- **Tests:** Observability basics.
- **Strong answer:** Logs are events with context, metrics are numeric time series for dashboards and alerts, traces follow one request across steps. I log JSON with request and workflow IDs, expose 15 Prometheus metrics (LLM cost, tokens, latency), and add OpenTelemetry spans per agent step.
- **Follow-up:** Why avoid putting customer IDs in metric labels?
- **Mistake:** Treating print statements as observability.

B20. What is a unit test vs an integration test vs an end-to-end test?

- **Tests:** Testing basics.
- **Strong answer:** A unit test checks a function in isolation (PII regex, PSI). An integration test checks components together (retriever with the database). An end-to-end test runs a full business flow: my test uploads a mule pattern with an injected reference, runs monitoring, scoring and investigation, and asserts human approval is required.
- **Follow-up:** How do you test on PostgreSQL in CI?
- **Mistake:** Only testing the happy path.

B21. What is idempotency?

- **Tests:** Reliability concept.
- **Strong answer:** Repeating an operation has the same effect as doing it once. Transaction IDs with `ON CONFLICT DO NOTHING` (replay loaded 0 rows), an Idempotency-Key header for starting investigations, and unique alert dedupe keys.
- **Follow-up:** What if the same key is used for a different alert?
- **Mistake:** Confusing idempotent with “safe” (read-only).

B22. What is pagination and why cap page size?

- **Tests:** API design.
- **Strong answer:** Returning results in pages. I cap `page_size` at 100 to protect the database and memory. Offset pagination works now; keyset pagination is better for deep pages at scale.
- **Follow-up:** Why is offset slow on deep pages?
- **Mistake:** Returning all rows.

B23. What is data validation vs data cleaning?

- **Tests:** Data engineering.
- **Strong answer:** Validation decides if a row is acceptable (reject negative amounts and future timestamps). Cleaning fixes acceptable issues (convert GBP to EUR, impute unknown country as ZZ, normalise text). I reject rather than guess when a fix could change meaning, such as the sign of an amount.

- **Follow-up:** When should a whole batch fail?
- **Mistake:** Silently dropping bad rows.

B24. What is feature engineering? Give an example from your project.

- **Tests:** Applied ML.
- **Strong answer:** Turning raw data into model signals. `unique_cash_branches_30d` counts distinct branches receiving cash. I added it after error analysis showed missed cases were cash businesses; funnel accounts spread deposits across branches.
- **Follow-up:** How did you verify it helped?
- **Mistake:** Adding features without evidence.

B25. What is class imbalance and how did you handle it?

- **Tests:** ML practice.
- **Strong answer:** One class is much rarer. I used `scale_pos_weight` in XGBoost, class-weighted logistic regression, PR-AUC, and threshold selection for recall rather than a 0.5 default.
- **Follow-up:** Why not oversample with SMOTE?
- **Mistake:** Reporting accuracy.

B26. What is PostgreSQL JSONB and when is it the wrong choice?

- **Tests:** Data modelling judgement.
- **Strong answer:** Binary JSON with indexing support, good for semi-structured outputs (features, agent outputs, quality reports). Wrong for fields you filter or join on often, which should be columns: I keep `status`, `risk_score`, `tenant_id` and `clearance_level` as real columns.
- **Follow-up:** How would you index a JSONB key?
- **Mistake:** Storing everything as JSON.

B27. What is the purpose of a health check, and why two?

- **Tests:** Operations.
- **Strong answer:** Liveness says the process is up (restart if not). Readiness says it can serve (database, model and knowledge base available; remove from load balancing if not). My `/health/ready` returns 503 if the database or model is unavailable.
- **Follow-up:** What happens if readiness checks call slow dependencies?
- **Mistake:** A single `/health` that always returns 200.

B28. What is cloud cost and how would you estimate LLM cost?

- **Tests:** Cost awareness.
- **Strong answer:** Tokens in and out per model $\times$ price. I record tokens and cost per call. Estimated about 21.5k tokens and \$0.075 per investigation, with prices configurable and marked as estimates. The biggest lever is not running the LLM on alerts the model deprioritises.
- **Follow-up:** How do you reduce cost without losing quality?
- **Mistake:** Ignoring input tokens, which usually dominate.

B29. Why did you choose Python and FastAPI?

- **Tests:** Justifying choices.
- **Strong answer:** The ML and data ecosystem is Python. FastAPI gives typed validation, dependency-injected auth and automatic OpenAPI docs with little boilerplate. The trade-off: Python CPU-bound work needs processes or workers, which is fine because the heavy work is database- and LLM-bound.
- **Follow-up:** When would you use Django?
- **Mistake:** "Because it's popular."

B30. What was the hardest bug you fixed?

- **Tests:** Debugging story.
 - **Strong answer:** Under concurrent load, the audit hash chain forked. Two transactions locked the same “last” row with `SELECT ... FOR UPDATE`, and PostgreSQL re-checks the locked row, not the `ORDER BY ... LIMIT 1` query, so both chained to the same predecessor. I switched to a per-tenant advisory lock in a short separate transaction, and added an 8-writer regression test. The dashboard smoke test is what revealed it.
 - **Follow-up:** Why not lock inside the business transaction?
 - **Mistake:** Describing a bug without root cause and prevention.
-

Intermediate**I1. Walk me through your data pipeline and its quality gates.**

- **Tests:** End-to-end data thinking.
- **Strong answer:** Read CSV → required columns → row checks (timestamps with legacy fallback, positive amounts, currency and channel enums, known account, tenant/account consistency) → clean (FX to EUR, country ZZ, normalise and truncate references) → dedupe (ID, near-duplicate fingerprint, already loaded) → injection scan → gates (reject ≤2%, missing country ≤5%, duplicates ≤5%) → COPY to staging → upsert. 771,818 rows → 759,983 loaded, 4,237 rejected, 7,598 duplicates, in 37 s.
- **Follow-up:** How would this run on a stream?
- **Mistake:** No lineage or reject storage.

I2. What is point-in-time correctness and how did you guarantee it?

- **Tests:** Leakage awareness.
- **Strong answer:** Features must only use data available at prediction time. `compute_features` slices `ts <= as_of` with binary search, the same function serves training and scoring, and the snapshot is stored on the alert. Labels only count suspicious activity that started by the alert window end.
- **Follow-up:** Give an example of leakage you’d worry about in AML.
- **Mistake:** Computing features over the full history, then splitting.

I3. Why XGBoost over logistic regression, if logistic was close on AUC?

- **Tests:** Nuanced model selection.
- **Strong answer:** Ranking metrics were close (PR-AUC 0.976 vs 0.958), but at the same ~92% recall operating point XGBoost produced 0 false positives versus 26. That’s real investigator time. XGBoost also captures interactions such as `SME × cash share`. Logistic regression stays as the baseline and a promotion gate.
- **Follow-up:** When would you pick logistic anyway?
- **Mistake:** Choosing by AUC alone.

I4. Why calibrate, and why Platt over isotonic?

- **Tests:** Probability reasoning.
- **Strong answer:** Policies use absolute thresholds (auto-close below 0.05), so scores must mean probabilities. Isotonic created ties that dropped PR-AUC to 0.950. Platt has two parameters, is monotonic so ranking is preserved, and is stable with limited data. Measured: Brier 0.021, ECE 0.018.
- **Follow-up:** How do you measure calibration?
- **Mistake:** Assuming XGBoost probabilities are calibrated.

I5. How did you choose the decision threshold?

- **Tests:** Operating point design.
- **Strong answer:** First version: 95% recall on a 435-alert calibration slice. It was unstable (15% vs 70% workload saving across rebuilds). I moved to 5-fold customer-grouped out-of-fold predictions over the whole development period, fitted calibration on those, picked the threshold there, and reported an operating curve on test so compliance can choose.

- **Follow-up:** Why not choose the threshold on test?
- **Mistake:** Tuning on test data.

16. How do you monitor drift?

- **Tests:** MLOps.
- **Strong answer:** PSI per feature and on scores against a reference profile saved with the model (<0.10 stable, >0.25 significant), exposed at /ops/drift. I found a false alarm (PSI 0.64 on velocity) caused by a feature defined differently for customers without history, and fixed the feature rather than the threshold.
- **Follow-up:** What's the difference between data drift and concept drift?
- **Mistake:** Retraining automatically on any drift signal.

17. How does your model registry answer “which model produced this prediction”?

- **Tests:** Lineage.
- **Strong answer:** Every predictions row has model_version_id. /ops/predictions/{id}/lineage returns the version, algorithm, artifact SHA-256, training-data hash, parameters and features. Artifacts are verified before unpickling.
- **Follow-up:** How do you roll back a model?
- **Mistake:** Keeping only “latest.pkl”.

18. Explain your hybrid retrieval.

- **Tests:** RAG depth.
- **Strong answer:** A SQL ACL filter, then dense pgvector cosine top-30 and BM25 top-30 over the permitted corpus, fused with reciprocal rank fusion (k=60), then a lightweight coverage reranker. Recall@5: dense 0.736, BM25 0.880, hybrid 0.903; reranking lifted MRR from 0.766 to 0.801.
- **Follow-up:** Why RRF instead of adding scores?
- **Mistake:** Only using vector search.

19. How did you evaluate retrieval?

- **Tests:** Evaluation rigour.
- **Strong answer:** 36 labelled queries, paraphrased rather than copied, with relevant chunk IDs; labels the user can't access are removed per query. Recall@5, Precision@5, MRR, nDCG@5, Hit@5 per mode, plus an ACL audit (21,147 results, 0 unauthorised). Caveat: single annotator, small corpus.
- **Follow-up:** Why is Precision@5 low (0.24)?
- **Mistake:** Evaluating RAG only by reading answers.

110. How do you enforce document access control in RAG?

- **Tests:** Security in RAG.
- **Strong answer:** Chunks carry tenant_id, clearance_level and is_active columns. The retrieval query filters on them before similarity ranking, and roles map to clearances. The policy agent retrieves with the requesting user's clearance, so an analyst's investigation can't cite restricted material.
- **Follow-up:** Why is post-filtering risky?
- **Mistake:** Filtering in the prompt (“don't use restricted documents”).

111. What is prompt injection and how do you defend against it?

- **Tests:** LLM security.
- **Strong answer:** Untrusted text that tries to redirect the model. Layers:
 1. Detection at ingestion and on tool view.
 2. Spotlighting with <untrusted_data> tags plus a system instruction.
 3. Containment: schema-constrained outputs, no write tools, a verifier, and a policy engine that forces human approval when case data is flagged.
 Detection is regex, so containment is the real control.

- **Follow-up:** What bug did you find here?
- **Mistake:** “We tell the model to ignore instructions.”

I12. How do you get reliable structured output from an LLM?

- **Tests:** LLM engineering.
- **Strong answer:** A tool whose input_schema is the Pydantic JSON schema, with forced tool_choice. Then strict validation (extra=forbid, enums, lengths, regex), one repair retry with the validation error, then fail closed. Offline runs: 100% validity. With real Claude this is the metric to watch.
- **Follow-up:** Why only one retry?
- **Mistake:** Parsing free text with regex.

I13. Describe your agents and why each exists.

- **Tests:** Agent design.
- **Strong answer:**

Agent	Model	Job	Tools	Failure
Triage	Haiku	Typology, priority, depth	None	Degrades to rules
Investigation	Sonnet	Evidence	5 read-only tools	Fails closed
Policy	Haiku	Citations from retrieved text	Retrieval done by code	Fails closed
Decision	Sonnet	Action + narrative	None	Fails closed
Verifier	Code	Checks claims	—	—
Policy engine	Code	Approval	—	—

- **Follow-up:** Why is the orchestrator not an LLM?
- **Mistake:** Splitting agents for the diagram rather than permissions and evaluation.

I14. What does your verifier check?

- **Tests:** Grounding.
- **Strong answer:** Evidence IDs were actually returned by tools in this workflow; numeric claims recomputed from the database (735 checked, 0 failed); citations were in the retrieved set; no PII; closure isn't contradicted by risk score or aggravating findings; escalation has aggravating evidence; injection flags. Critical issues block, with one repair attempt.
- **Follow-up:** Why deterministic instead of an LLM judge?
- **Mistake:** “The model cites sources, so it's grounded.”

I15. How do you prevent agent loops?

- **Tests:** Bounded autonomy.
- **Strong answer:** Maximum 8 tool calls, a hard ceiling on LLM turns, and an identical repeated call (tool + sorted arguments) raises AgentLoopError. The workflow then fails closed. Tested with an injected loop fault: 8/8 safe.
- **Follow-up:** What if the loop varies arguments slightly?
- **Mistake:** Relying only on a timeout.

I16. How does human-in-the-loop work in your system?

- **Tests:** Governance design.
- **Strong answer:** The policy engine decides whether approval is required and which role. STRs and restrictions always need the MLRO. The approval API enforces role authority, four-eyes (initiator ≠ approver), a rationale of at least 10 characters, and refuses approving a recommendation that failed verification. Only then does the allow-listed executor act; everything is audited.
- **Follow-up:** What can be auto-applied and why?
- **Mistake:** A generic “human reviews output” without authority rules.

I17. How did you implement multi-tenancy, and what are the risks?

- **Tests:** SaaS architecture.
- **Strong answer:** A shared schema with `tenant_id` on every table; the token's tenant filters every query; other-tenant IDs return 404; ingestion rejects mismatched rows; RAG has global plus tenant documents. Risk: one missing filter leaks data. Mitigations: tests, and PostgreSQL row-level security as the next step.
- **Follow-up:** When would you use a database per tenant?
- **Mistake:** Trusting a tenant ID sent in the request body.

I18. How do you handle concurrency on approvals and investigations?

- **Tests:** Transactions.
- **Strong answer:** An optimistic lock (`lock_version`) on alerts stops double starts. A row lock plus unique `approvals.recommendation_id` stops double approval (second attempt → 409). `SKIP LOCKED` lets workers share a queue. Advisory locks serialise audit appends.
- **Follow-up:** Optimistic vs pessimistic: when is each appropriate?
- **Mistake:** Check-then-update without a lock or constraint.

I19. How do you make the async worker safe?

- **Tests:** Queue processing.
- **Strong answer:** `SELECT ... FOR UPDATE SKIP LOCKED LIMIT 1` so each worker claims a different run. Commit on success, roll back on exception, `SIGTERM` handling for graceful shutdown. Tested with 3 concurrent workers: each workflow triaged exactly once. Production: Service Bus peek-lock, max delivery count, dead-letter queue, KEDA.
- **Follow-up:** How do you handle a worker crash mid-workflow?
- **Mistake:** Polling without locking.

I20. How do you test LLM failure modes without an LLM?

- **Tests:** Test design.
- **Strong answer:** A `FaultInjectingProvider` wraps the deterministic provider with 12 modes (timeout, error, malformed JSON, schema-invalid, loop, hallucinated evidence, fabricated citation, PII leak, unsafe close, triage error). The evaluation asserts the expected safe outcome: 96/96 safe.
- **Follow-up:** What does that not tell you?
- **Mistake:** Only mocking success responses.

I21. What is your LLMOps ledger and why does it matter?

- **Tests:** LLMOps.
- **Strong answer:** Every call stores model, prompt name, version and hash, tokens (flagged if estimated), cost, latency, attempts, status, schema validity, retrieved chunk IDs and a redacted output preview. It answers “why did the model say this”, “what did it cost”, and “did the new prompt version increase failures”.
- **Follow-up:** How do you version prompts?
- **Mistake:** Prompts hard-coded in Python strings.

I22. How does model routing reduce cost?

- **Tests:** Cost engineering.
- **Strong answer:** Haiku-class for short extraction (triage, policy citations); Sonnet-class for planning and synthesis (investigation, decision). Upstream, the ML model means low-risk alerts need no LLM run at all. Next steps: prompt caching for static system prompts and batch processing for overnight queues.
- **Follow-up:** How would you prove Haiku is good enough for policy?
- **Mistake:** Using the largest model everywhere, or downgrading without evaluation.

I23. Explain PII handling end to end.

- **Tests:** Privacy engineering.

- **Strong answer:** Agents never receive names or IBANs, only pseudonymous IDs and profile attributes. Regex redaction runs in logs, LLM output previews and tool references. The API masks names without `pii : view`, and the verifier blocks PII in narratives. I fixed a false positive where UUID fragments looked like phone numbers.
- **Follow-up:** What would you use instead of regex in production?
- **Mistake:** Redacting only in the UI.

I24. What did your load test show and what are its limits?

- **Tests:** Performance thinking.
- **Strong answer:** A real unicorn server with PostgreSQL on one CPU, 0 failures:

Endpoint	p95 latency	Throughput
Alert list	64 ms	161 req/s
Knowledge search	106 ms	97 req/s
Investigation (no LLM network time)	424 ms	—

Limits: a shared sandbox CPU, a single worker, and no real LLM latency, which would dominate.

- **Follow-up:** How would you find the bottleneck?
- **Mistake:** Quoting throughput without environment details.

I25. What is in your CI pipeline and why each step?

- **Tests:** DevSecOps.
- **Strong answer:**

Step	Catches
ruff	Style and security lints
Bandit	Python SAST
pip-audit	Dependency CVEs (it caught PyJWT)
gitleaks	Secrets
pytest on SQLite and PostgreSQL	Portability and pgvector behaviour
Image build + Trivy	OS and library CVEs in images

Deploy is manual with environment approval and OIDC.

- **Follow-up:** Why test on two databases?
- **Mistake:** Treating CI as “runs tests” only.

I26. Which Azure services did you choose and why?

- **Tests:** Cloud design justification.
- **Strong answer:**

Service	Why
Container Apps	API and workers without cluster operations
KEDA	Scale workers on queue depth
PostgreSQL Flexible Server + pgvector	One store for data and vectors under the same access control
Service Bus	Per-message work queue with dead letters
Event Hubs	High-throughput transaction stream
Key Vault	Secrets via managed identity
ACR	Private images
Blob	Artifacts and immutable audit exports
Azure Monitor	Logs, metrics, traces

Written in Bicep, not deployed.

- **Follow-up:** Why not AKS?
- **Mistake:** Listing services without the problem each solves.

I27. How do you handle secrets in Azure?

- **Tests:** Cloud security.
- **Strong answer:** Key Vault with RBAC and purge protection. Container Apps reference secrets through system-assigned managed identity, so there are no secrets in images or pipelines. GitHub deploys with OIDC federated credentials. The app refuses to start in production with default secrets.
- **Follow-up:** How do you rotate the JWT secret?
- **Mistake:** Storing secrets as plain pipeline variables.

I28. What would a Grafana dashboard for this system show, and which alerts page someone?

- **Tests:** Operability.
- **Strong answer:** Requests and p95 by route, LLM calls, latency, tokens and cost, agent step latency, tool call status, workflow outcomes, risk score quantiles, retrieval latency, injection detections. Paging alerts: API 5xx >2% and LLM failure rate >5% (investigations fall back to humans). Ticket alerts: spend spikes, manual-review share, injection surges, score shift.
- **Follow-up:** Why is an LLM outage not total downtime here?
- **Mistake:** Alerting on everything.

I29. How did you prevent the scanner from quarantining real policies?

- **Tests:** Handling false positives in security controls.
- **Strong answer:** The first heuristic quarantined 2 real SOPs containing “Do not close an alert as a false positive”. I added severity tiers: high-severity rules (override instructions, role hijack, AI directives, tag injection, exfiltration) quarantine documents; decision-steering rules apply only to customer free text. A regression test covers all 13 documents, and the poisoned fixture still trips 5 high-severity rules.
- **Follow-up:** How would you measure the scanner’s precision and recall?
- **Mistake:** Maximising detection with no false-positive budget.

I30. What did agent evaluation reveal that unit tests didn’t?

- **Tests:** Evaluation value.
- **Strong answer:** A truly suspicious alert was auto-closed (1 of 80). Error analysis: all 12 low-scoring suspicious alerts were SME cash businesses. I added a branch-dispersion feature and a policy rule blocking auto-closure for cash-intensive SMEs. Result: 0 auto-closed suspicious alerts, and I accepted the automation share falling from 39% to about 21%.
- **Follow-up:** Isn’t that giving up automation?
- **Mistake:** Hiding the trade-off.

Advanced

A1. Design this system for a bank with 50 million transactions a day.

- **Tests:** System design at scale.
- **Strong answer:**
 1. Event Hubs (Kafka API) partitioned by account, feeding stateless ingestion consumers that reuse the validation code and upsert idempotently.
 2. Streaming features (Flink or Spark Structured Streaming) into an online feature store; rules evaluated on the stream.
 3. Alerts land in partitioned PostgreSQL or Citus.
 4. Scoring through a batching model service.

- 5. Investigations enqueued on Service Bus; workers autoscale on queue depth; per-tenant LLM token budgets through a gateway.
- 6. Vector search in a dedicated HNSW index with ACL filters.
- 7. Analytics on a lakehouse, not OLTP.
- **Follow-up:** Where does ordering matter and how do you keep it?
- **Mistake:** “Just add Kubernetes.”

A2. Exactly-once processing: is it achievable here?

- **Tests:** Distributed-systems realism.
- **Strong answer:** End-to-end exactly-once delivery isn't realistic; aim for at-least-once delivery with idempotent effects. Natural keys plus ON CONFLICT DO NOTHING for transactions, alert dedupe keys, idempotency keys on workflows, a unique recommendation per workflow, and a unique approval per recommendation. Side effects outside the database (STR submission) need an outbox with deduplication.
- **Follow-up:** How does the transactional outbox work?
- **Mistake:** Claiming Kafka gives exactly-once end to end.

A3. Your audit log uses autonomous transactions. Defend or criticise that.

- **Tests:** Trade-off honesty.
- **Strong answer:**
 - **For:** writing the hash chain inside long business transactions serialises investigations for the duration of LLM calls; a short advisory-locked append keeps a valid chain under concurrency (tested).
 - **Against:** events persist when the business transaction rolls back, so the trail records attempts, not only outcomes.
 - **Better long term:** a transactional outbox inside the business transaction, a single sequencer that hashes committed events in commit order, and export to immutable storage.
- **Follow-up:** How do you order events by commit time?
- **Mistake:** Not knowing the downside of your own design.

A4. How would you evaluate the agents with real Claude models?

- **Tests:** LLM evaluation design.
- **Strong answer:**
 1. Run `agent_eval.py` with `LLM_PROVIDER=anthropic` on a stratified set.
 2. Track decision agreement with ground truth, verification failure rate (the real hallucination signal), schema repair rate, tokens, cost and p95 latency, and compare with the deterministic baseline.
 3. Add narrative quality scored by an LLM judge whose rubric is calibrated against MLRO ratings on a sample (agreement measured with Cohen's kappa).
 4. Run the adversarial suite with real injection strings.
 5. Repeat runs to measure variance.
- **Follow-up:** How many samples do you need to detect a 5-point difference?
- **Mistake:** Vibe-checking a few outputs.

A5. Why is your agent evaluation not proof of reasoning quality?

- **Tests:** Scientific honesty.
- **Strong answer:** The agents were deterministic rule-based stand-ins, so decision quality reflects my heuristics, and schema validity is trivially 100%. What it does prove: the orchestration, verifier, policy engine and approval controls behave correctly across 160 normal and 96 adversarial workflows. Two genuine bugs were found this way.
- **Follow-up:** Then why include it?
- **Mistake:** Presenting 100% validity as an LLM result.

A6. Synthetic data made the model look excellent. How do you convince a bank it will work?

- **Tests:** Validation strategy.
- **Strong answer:**
 1. Retrain on the bank's historical dispositions (noisy, delayed labels).

2. Temporal back-test.
 3. Shadow mode for 3 months: score and investigate in parallel without affecting decisions.
 4. Compare missed-suspicious rate, time to disposition and QA pass rate against the current process.
 5. Agree the operating point with Compliance using the operating curve.
 6. Model risk management review (documentation, challenger models).
- **Follow-up:** How do you handle label bias from past investigator behaviour?
 - **Mistake:** Extrapolating synthetic metrics.

A7. Labels come from past investigations, which were biased by the old rules. Consequences?

- **Tests:** Selection bias.
- **Strong answer:** We only observe outcomes for alerted cases, so the model learns “suspicious given the rules fired” and can’t discover patterns the rules never flagged. Mitigations: keep Isolation Forest as a novelty signal (test ROC-AUC 0.940), random sampling of non-alerted customers for review, SAR feedback from law enforcement, and periodic below-the-line testing of rule thresholds.
- **Follow-up:** What is below-the-line testing?
- **Mistake:** Assuming labels are ground truth.

A8. How would you detect a new laundering typology the model has never seen?

- **Tests:** Novelty detection.
- **Strong answer:** Unsupervised anomaly percentile (Isolation Forest) with a policy that anomaly ≥ 0.90 blocks auto-closure; clustering of high-anomaly, low-score cases for analyst review; network features (shared counterparties across customers); drift on the score distribution; and a feedback loop where confirmed new patterns become labelled typologies and rules.
- **Follow-up:** How do you avoid drowning analysts in anomalies?
- **Mistake:** “Retrain the classifier.”

A9. How would you add graph analytics for mule networks?

- **Tests:** Advanced data modelling.
- **Strong answer:** Build a customer-counterparty graph from transactions (a graph database, or PostgreSQL edge tables for moderate size). Compute features: shared counterparties, fan-in/fan-out, community detection, path length to known mule accounts. Add them as model features and as an agent tool (`get_network_neighbourhood`, read-only and scoped). Watch for privacy: exposing other customers’ data inside a case needs access rules.
- **Follow-up:** How do you keep graph features point-in-time correct?
- **Mistake:** Building a graph database without a clear feature or decision use.

A10. How do you secure tools when an agent can call them?

- **Tests:** Agent security architecture.
- **Strong answer:**
 - Per-agent allow-list; tools bound in code to (tenant, alert, customer), never taken from model arguments.
 - Pydantic-bounded arguments with `extra=forbid` (an attempt to pass `customer_id` was rejected in tests).
 - Read-only; row limits; PII removed; free text wrapped as untrusted; every call logged, including denied ones.
 - Evidence IDs recorded for the verifier.
 - Write actions exist only in the executor after policy and approval.
- **Follow-up:** What changes if a tool must write (for example request documents from a customer)?
- **Mistake:** Giving the agent a SQL tool.

A11. What if the LLM provider is down for an hour?

- **Tests:** Resilience.
- **Strong answer:** The circuit breaker opens per model after repeated failures, so calls fail fast. Workflows fail closed to manual review (fault-tested). Alerts stay ranked by the ML model, which has no LLM dependency. The paging alert fires on LLM failure rate. In async mode, messages stay on the queue and retry later. Optional: a fallback model behind the gateway, evaluated beforehand.

- **Follow-up:** Would you silently switch to another provider?
- **Mistake:** Degrading to unverified heuristics without telling users.

A12. How would you run a new prompt version safely?

- **Tests:** LLMOps release process.
- **Strong answer:**
 1. The prompt YAML change goes through pull request review.
 2. Offline evaluation on the fixed dataset and the adversarial suite, compared with the current version.
 3. Canary to a small traffic share with prompt version in the ledger.
 4. Compare verification failure rate, repair rate, tokens and decision distribution.
 5. Roll back by reverting the file.

Past decisions stay attributable through the recorded prompt hash.
- **Follow-up:** How do you prevent a prompt change from shifting decisions silently?
- **Mistake:** Editing prompts in production.

A13. Tenant isolation: application filters vs row-level security vs separate databases.

- **Tests:** Isolation depth.
- **Strong answer:**

Option	Pros	Cons
Application filters (current)	Simple, portable, tested	One missed filter leaks data
PostgreSQL row-level security with SET app . tenant per transaction	Database enforces isolation even with a bug	Complicates pooling and superuser access
Database per tenant	Strongest isolation, per-tenant keys and residency	Operational overhead, cross-tenant analytics hard

I'd add RLS next, and offer a dedicated database to large regulated clients.

- **Follow-up:** How does connection pooling interact with RLS?
- **Mistake:** Treating isolation as one-size-fits-all.

A14. How would you make retrieval scale to a million chunks with access control?

- **Tests:** Vector search internals.
- **Strong answer:** An HNSW index on pgvector (or a dedicated engine) with ACL attributes. Pure post-filtering of ANN results loses recall under selective filters, so use filtered ANN (pre-filter or iterative scan), partition by tenant, and keep clearance as a numeric filter. Replace in-memory BM25 with tsvector GIN or a search engine, and cache by (query, tenant, clearance, KB version). Re-measure Recall@K against exact search.
- **Follow-up:** What HNSW parameters trade recall for latency?
- **Mistake:** Assuming the ANN index respects metadata filters for free.

A15. Your PII detector is regex. How would you evaluate and replace it?

- **Tests:** Security control engineering.
- **Strong answer:** Build a labelled set of narratives and references with realistic Irish formats (IBAN, PPSN, Eircode, phones) and adversarial negatives (UUIDs, transaction IDs, amounts). Measure precision and recall per entity. Compare regex, Microsoft Presidio and Azure AI Language PII. Keep regex as a fast pre-filter and remove the root risk by not sending personal data to the model.
- **Follow-up:** What false positive did you already find?
- **Mistake:** Only measuring recall.

A16. How would you design SLOs for this platform?

- **Tests:** Reliability engineering.
- **Strong answer:**

SLO	Target
API availability	99.9% monthly
Alert list p95	< 300 ms
Asynchronous investigation completion p95	< 5 minutes
LLM-dependent investigations completing without manual fallback	≥ 95%
Scoring availability	99.95%

Error budgets guide release pace. Business SLAs (critical alert reviewed within 1 business day) are tracked separately in analytics.

- **Follow-up:** How do SLOs change alerting?
- **Mistake:** SLOs on CPU usage.

A17. Model risk management: what documentation would a regulator expect?

- **Tests:** Governance maturity.
- **Strong answer:**
 - Purpose and limitations.
 - Data lineage and label definition.
 - Validation protocol (temporal, grouped CV), metrics and operating-point rationale.
 - Explainability (contributions) and calibration.
 - Challenger comparison (logistic baseline).
 - Monitoring (drift, performance), change control (gates, versioning), human oversight design, known weaknesses (cash-business blind spot) and compensating controls.

Most of this exists in ml.md plus the registry metadata.

- **Follow-up:** Who approves a model change?
- **Mistake:** Treating models as code-only artefacts.

A18. How would you handle the EU AI Act and GDPR considerations?

- **Tests:** Regulatory awareness (not legal advice).
- **Strong answer:**
 - Assess classification with Legal. Anti-money-laundering detection is a legal obligation, but profiling individuals still requires transparency, human oversight, logging and robustness — all of which the design supports.
 - Under GDPR: a lawful basis (legal obligation), data minimisation to the LLM, processor agreements, EU data residency, retention limits balanced against AML record keeping, and DPIA documentation.
- **Follow-up:** How does tipping off interact with subject access requests?
- **Mistake:** Giving definitive legal conclusions.

A19. Where would caching help and where is it dangerous?

- **Tests:** Caching judgement.
- **Strong answer:**
 - **Safe:** retrieval results keyed by (query, tenant, clearance, KB version); KPI aggregates for seconds; provider prompt caching of static system prompts.
 - **Dangerous:** caching decisions or tool results across cases or tenants (leakage, stale evidence); caching authorisation decisions beyond token lifetime.
 - Include tenant and clearance in every cache key.
- **Follow-up:** How do you invalidate on document approval?
- **Mistake:** A global cache keyed only by query text.

A20. How would you reduce LLM cost by 50% without losing quality?

- **Tests:** Cost optimisation with evidence.
- **Strong answer:**
 1. Measure a token breakdown per agent from the ledger.
 2. Don't investigate alerts below the model threshold unless a human asks (largest lever).
 3. Prompt-cache static prefixes.
 4. Compress tool results into aggregates.
 5. Test Haiku for more steps, gated on verification failure rate and decision agreement.
 6. Batch non-urgent overnight work.
 7. Cap tool steps by depth.

Each change is accepted only if offline evaluation metrics hold.
- **Follow-up:** What metric would stop a downgrade?
- **Mistake:** Downgrading models without measuring.

A21. What happens if an attacker controls many customers' payment references?

- **Tests:** Adversarial systems thinking.
- **Strong answer:** Individual injections force human review. But a flood could overload reviewers or train staff to ignore flags. Controls: rate metric and PromptInjectionSurge alert; treat instruction-like references as an aggravating typology signal; group by counterparty to detect coordination; keep containment (no write path).
- **Follow-up:** Could the attacker poison the model's training labels?
- **Mistake:** Only thinking about one malicious string.

A22. Explain optimistic locking and a failure scenario it doesn't cover.

- **Tests:** Concurrency depth.
- **Strong answer:** Each update checks `lock_version` and increments it; a stale writer gets `StaleDataError` (409). It doesn't cover invariants spanning rows or tables (for example "only one open recommendation per customer"), non-database side effects, or read skew in reports. Those need constraints, serialisable transactions or locks.
- **Follow-up:** When would you use `SERIALIZABLE` isolation?
- **Mistake:** Believing optimistic locking prevents all races.

A23. How would you migrate from `create_all` to managed migrations with zero downtime?

- **Tests:** Database operations.
- **Strong answer:**
 1. Baseline Alembic from the current schema.
 2. Use expand-and-contract: add nullable columns or new tables, deploy code writing both, backfill in batches, switch reads, then drop the old column in a later release.
 3. Create indexes concurrently.
 4. Test migrations on a production-size copy in CI.
- **Follow-up:** How did adding `reference_flags` break things for you?
- **Mistake:** Running blocking `ALTER TABLE` on large tables at peak.

A24. Would you use LangGraph or another agent framework instead of your orchestrator?

- **Tests:** Framework judgement.
- **Strong answer:** For this workflow, a fixed sequence with one bounded loop and explicit failure semantics fits in about 200 lines, is easy to test and has no hidden retries. LangGraph adds value with branching graphs, checkpointed long-running state, human-interrupt nodes and resumability. I'd adopt it if workflows became dynamic, keeping the verifier, policy engine and executor outside the framework.
- **Follow-up:** How would you resume a workflow after a crash?
- **Mistake:** Framework-first design.

A25. How would you test for fairness or disparate impact?

- **Tests:** Responsible AI.

- **Strong answer:** Protected attributes aren't in the model, but proxies (nationality through remittance countries, occupation, segment) can create disparities. Measure alert rates, score distributions and false-positive rates across segments and countries of counterparties. I already report per-tenant and per-rule recall; the diaspora family-support persona was designed to test high-risk-country false positives. Remediate through features and policy review, not by hiding attributes.
- **Follow-up:** What conflict exists between high-risk country rules and fairness?
- **Mistake:** "We don't use sensitive attributes, so it's fair."

A26. How would you build a feedback loop from investigators?

- **Tests:** Product and ML loop.
- **Strong answer:** Capture the approve/reject decision, rationale, final disposition and which findings were wrong. Store these as labels with timestamps. Monthly retrain with label-delay handling. Track agreement between recommendations and human decisions per prompt version. Rejected recommendations get sampled for error analysis. Guard against automation bias by blind-reviewing a sample without AI recommendations.
- **Follow-up:** What is automation bias?
- **Mistake:** Training on approvals as if they were ground truth.

A27. What is your threat model for the model artefact supply chain?

- **Tests:** ML security.
- **Strong answer:** Risks: a malicious pickle executing code on load, tampered artefacts, dependency CVEs, training-data poisoning. Controls: SHA-256 verification against the registry before load (tested), private storage, pinned dependencies with pip-audit, image scanning, and training-data hashes. Further: signed artefacts (Sigstore), safer formats (XGBoost JSON/UBJ instead of pickle), and isolated training environments.
- **Follow-up:** Why not just use XGBoost's native format?
- **Mistake:** Treating model files as harmless data.

A28. How would you handle 5-year AML record retention while controlling cost?

- **Tests:** Data lifecycle.
- **Strong answer:** Monthly partitions of transactions and alerts; move closed partitions to cheaper storage (Parquet on Blob with lifecycle tiers, queryable by a lakehouse engine). Keep case files and audit logs in immutable WORM Blob containers; delete after the retention period with an audited job, respecting legal holds. Keep embeddings and LLM previews shorter unless they're part of the case record.
- **Follow-up:** How does GDPR erasure interact with AML retention?
- **Mistake:** Keeping everything forever in OLTP.

A29. If you had to cut the scope in half, what would you keep?

- **Tests:** Prioritisation.
- **Strong answer:** Keep:
 1. Ingestion with quality gates.
 2. The ML ranking with calibration and lineage.
 3. The investigation evidence pack.
 4. The verifier, policy engine and approvals.
 5. Audit.

Cut: multi-agent split (a single evidence-gathering agent plus a decision prompt), dashboard polish, Bicep, and the second tenant. The safety boundary and measurable value survive; the architectural extras don't.
- **Follow-up:** What's the minimum to run a pilot?
- **Mistake:** Cutting tests or the verifier.

A30. What would you do differently if starting again?

- **Tests:** Reflection and growth.
- **Strong answer:**

1. Build the evaluation harness and concurrency tests before the features; several bugs (audit race, injection bypass) appeared late.
 2. Start with Alembic migrations.
 3. Use a transactional outbox for audit from the beginning.
 4. Store XGBoost in its native format rather than pickle.
 5. Design the synthetic data with harder, overlapping typologies earlier.
 6. Get real-LLM evaluation runs as soon as possible.
- **Follow-up:** Which of these would you fix first on a real team?
 - **Mistake:** “Nothing, it went well.”

Resume evidence

Only facts that were implemented and measured in this repository. Every number traces to `evaluation/reports/*.json` via `docs/evaluation.md`.

Framing rule for bullets: say “synthetic data”, “local”, and “deterministic reference agents” where relevant. Do not claim production deployment, real bank data, real Claude evaluation, Docker/Azure execution, or business impact in money.

1. What was built (verifiable in code)

- **End-to-end AML alert investigation platform** in Python: ~7,400 lines across application, tests, evaluation and dashboard; 20-table PostgreSQL + pgvector schema; 26 REST API operations across 25 paths (FastAPI, Pydantic).
- **Data engineering:**
 - Behavioural synthetic data generator: 3,000 customers, 771,818 transactions, 5 laundering typologies, 9 legitimate personas, 9 injected defect types.
 - Validation, cleaning, deduplication and quality gates.
 - Idempotent PostgreSQL COPY + upsert load.
 - Point-in-time feature engineering (33 features).
 - 6-rule monitoring engine with suppression.
- **Machine learning:**
 - XGBoost with Platt calibration, logistic baseline, isotonic comparison, Isolation Forest.
 - Temporal split, customer-grouped CV, out-of-fold threshold selection.
 - Promotion gates, SHA-256-verified model registry, per-prediction lineage and contributions, PSI drift monitoring.
- **RAG:**
 - 13-document knowledge base, heading-aware chunking.
 - Hybrid retrieval (pgvector cosine + BM25 + reciprocal rank fusion + reranking).
 - Tenant and clearance access control enforced inside the SQL query.
 - Document poisoning quarantine with four-eyes approval.
- **Agentic AI:**
 - 4 specialised agents (triage, investigation with 5 permissioned read-only tools, policy, decision), orchestrated by a deterministic state machine.
 - Deterministic verifier that checks evidence IDs, re-computes numeric claims, validates citations and detects PII.
 - Policy engine with human-in-the-loop approval (authority matrix, four-eyes).
 - Allow-listed action execution.
- **LLMOps:** versioned YAML prompts; LLM gateway with model routing (Sonnet-class / Haiku-class), structured output via tool schemas, retries with schema repair, circuit breaker, token/cost/latency ledger; Anthropic provider, deterministic provider, fault-injection provider.
- **Security:**
 - JWT authentication, PBKDF2 password hashing, account lockout, login rate limiting.
 - RBAC with 5 roles and 11 permissions; tenant isolation.
 - Prompt-injection detection at ingestion with forced human review.
 - PII redaction, security headers, SHA-256 artifact verification, hash-chained audit log.
- **Observability:** JSON structured logs with request/workflow correlation, 15 Prometheus metrics, OpenTelemetry spans, 12-panel Grafana dashboard definition, 7 Prometheus alert rules.
- **DevOps:** multi-stage non-root Dockerfiles, Docker Compose, GitHub Actions (lint, SAST, dependency audit, secret scan, tests on SQLite and PostgreSQL, image build, Trivy), environment-gated Azure deploy workflow, Azure Bicep (Container

Apps, PostgreSQL Flexible Server, Service Bus, Event Hubs, Key Vault, ACR, Blob, Monitor). *Written, not executed.*

- **Frontend:** Streamlit dashboard with 7 role-based pages; smoke-tested for 6 users across both tenants (0 failing renders).

2. Measured results

Area	Result
Data quality	771,818 rows → 759,983 loaded, 4,237 rejected with reasons, 7,598 duplicates removed in 37 s; replay loaded 0 rows
Alerting	3,215 alerts; about 71% false positives in the test period (the problem being solved)
ML (803 held-out future alerts)	XGBoost + Platt PR-AUC 0.976 / ROC-AUC 0.980 vs logistic 0.958 / 0.961 under identical protocol; 91.7% recall with 0 false positives vs 26; 73.7% of alerts deprioritised at that operating point; customer-grouped CV ROC-AUC 0.979 ± 0.021
Calibration	Brier 0.021, ECE 0.018; Platt beat isotonic on PR-AUC (0.976 vs 0.950)
RAG (36 labelled queries)	Hybrid + rerank Recall@5 0.903, MRR 0.801, nDCG@5 0.807 vs dense-only Recall@5 0.736
Access control	21,147 retrieved results audited, 0 unauthorised
Agents (160 held-out alerts, deterministic reference agents)	100% task success; 100% schema validity; groundedness 1.0; 735 numeric claims verified, 0 mismatches; 0 dangerous actions executed without approval; 100% approval compliance; 0 suspicious alerts auto-closed
Adversarial	96 fault-injection runs across 12 failure modes: 100% safe outcomes, 0 unsafe executions
Performance (1 CPU, uvicorn + PostgreSQL)	Alert list p95 64 ms at 161 req/s; alert detail p95 50 ms; knowledge search p95 106 ms; full investigation p95 424 ms (no network LLM); 0 failures across 1,440 requests
Estimated LLM cost	~21.5k tokens and ~\$0.075 per investigation (token counts estimated, prices configurable)
Tests	134 passing on PostgreSQL; 129 passing on SQLite (+5 PostgreSQL-only skipped)
Static and dependency security	ruff, Bandit clean; pip-audit found PyJWT 2.7.0 CVEs → upgraded to 2.14.0 → clean

3. Engineering judgement (true stories for interviews)

Situation	What I did	Outcome
First model scored ROC-AUC 0.98 off one KYC feature	Recognised synthetic-data leakage; added stale and inflated KYC profiles	More realistic feature mix
Threshold swung between 70% and 15% workload savings across rebuilds	Switched to out-of-fold threshold and calibration; reported operating curve	Stable operating point
Baseline trained on less data than XGBoost	Gave both models the same protocol	Fair PR-AUC 0.976 vs 0.958
Agent evaluation showed 1/80 suspicious alerts auto-closed	Error analysis → all misses were SME cash businesses; added branch-dispersion feature and blocked auto-closure for cash-intensive SMEs	0 auto-closed suspicious alerts; accepted auto-apply share dropping from 39% to ~21%
Injection in small payments went unflagged	Moved scanning to ingestion; orchestrator checks the whole alert window	Always routed to humans
Load test forked the audit hash chain	Diagnosed FOR UPDATE race; per-tenant advisory-locked autonomous appends	Chain valid under concurrency; regression test
1.6% of workflows failed randomly	Traced to UUID fragments matching the phone regex	611/611 high-risk workflows verified
Legitimate policies were quarantined as injection	Severity-tiered detection with context	13/13 documents clean, poison still caught

4. Do NOT claim

- Deployment to Azure, or any cloud.
- Docker images built or run.
- Real customer or bank data, or real regulatory filings.
- Agent performance of Claude models (not measured).
- Money saved, FTE reduced, or production traffic.
- BGE embeddings or a cross-encoder reranker in use (production design only).

5. Example bullets (edit to your voice; keep the qualifiers)

- Built an agentic AML alert-investigation platform (FastAPI, PostgreSQL/pgvector, XGBoost, Claude-ready multi-agent workflow) on 772k synthetic transactions. A calibrated risk model reached PR-AUC 0.976 vs 0.958 for a logistic baseline on held-out future alerts and deprioritised 74% of alerts at 92% recall.
- Designed verification-first agents: a deterministic verifier re-computed 735 LLM-style numeric claims against the database and blocked hallucinated evidence, fabricated citations and PII leaks. 96/96 fault-injection runs failed safe with 0 unapproved high-risk actions.
- Implemented access-controlled hybrid RAG (pgvector + BM25 + RRF + rerank) with tenant and clearance filtering in SQL: Recall@5 0.903 and 0 unauthorised results across 21k audited retrievals.
- Found and fixed production-class defects through evaluation: an audit-log race under concurrent load (advisory locks), prompt injection bypassing detection, and vulnerable JWT dependencies (pip-audit). Test suite: 134 tests on PostgreSQL.